Guide Complet du Projet TechKids Hub

Document de Référence pour les Équipes de Développement

Version 1.0

27 décembre 2025

Projet Académique

Ce document constitue le guide technique complet pour la réalisation du projet TechKids Hub dans le cadre de projets de fin d'études (PFE).

Table des matières

1 Vision et Positionnement du Projet **3**
1.1 Concept Central : L'Écosystème "Phygital" . . . 3
1.2 Les Différenciateurs Clés . . . 3
1.3 User Journey Unifié . . . 3
1.3.1 Pour le Parent . . . 3
1.3.2 Pour l'Enfant . . . 3

2 Architecture Globale **5**
2.1 Stack Technique . . . 5
2.1.1 Frontend . . . 5
2.1.2 Backend . . . 5
2.1.3 IA & Simulation . . . 5
2.2 Architecture en Microservices . . . 5
2.3 Organisation en Monorepo . . . 6

3 Schéma de Base de Données **7**
3.1 Domaine : Noyau (Users & Auth) . . . 7
3.2 Points Clés du Schéma . . . 7
3.2.1 Flexibilité Kit vs PhysicalItem . . . 7
3.2.2 Workflow de Génération de Contenu . . . 8
3.2.3 Progression Granulaire . . . 8

4 Répartition des Projets de Fin d'Études (PFE) et Spécifications IA **9**
4.1 PFE 1 : Plateforme E-Commerce & Gestion des Kits . . . 9
4.1.1 Implémentation IA : Algorithmes et Bibliothèques . . . 9
4.2 PFE 2 : Edu-Tracker (LMS & Suivi Pédagogique) . . . 10
4.2.1 Implémentation IA : Algorithmes et Bibliothèques . . . 10
4.3 PFE 3 : Virtual Lab (Simulation & Tutorat IA) . . . 11
4.3.1 Implémentation IA : Algorithmes et Bibliothèques . . . 11
4.4 PFE 4 : Génération de Contenu Pédagogique Automatisé . . . 12
4.4.1 Implémentation IA : Algorithmes et Bibliothèques . . . 12

5 Méthodologie de Validation et d'Implémentation des Composants IA **13**
5.1 Métriques d'Évaluation des Modèles . . . 13
5.1.1 Pour les systèmes de recommandation (PFE 1) . . . 13
5.1.2 Pour la prédiction des difficultés (PFE 2) . . . 13
5.1.3 Pour le Tutorat et la Génération (PFE 3 & 4) . . . 13
5.2 Feuille de Route pour l'Implémentation Technique . . . 14
5.2.1 Étape 1 : Collecte et Préparation des Données (Data Engineering) . . . 14

5.2.2 Étape 2 : Entraînement et Sélection du Modèle (Model Selection) . 14
5.2.3 Étape 3 : Intégration et Déploiement (Deployment) 14
5.2.4 Étape 4 : Monitoring et Boucle de Rétroaction (Feedback Loop) . . 14

6 Schéma de Base de Données Complet 15
6.1 Schéma Prisma Complet et Optimisé 15
6.2 Domaine : Noyau (Users & Auth) 15
6.3 Domaine : E-Commerce (PFE 1) 16

7 Points Clés du Schéma 18
7.1 Flexibilité Kit vs PhysicalItem 18
7.2 Workflow de Génération de Contenu 18
7.3 Progression Granulaire 18

8 Répartition des PFE et Responsabilités 19
8.1 PFE 1 : E-Commerce & Gestion des Kits 19
8.1.1 Périmètre 19
8.1.2 Endpoints Clés à Exposer 19
8.1.3 IA Intégrée 19
8.2 PFE 2 : Edu-Tracker (LMS & Suivi Pédagogique) 19
8.2.1 Périmètre 19

9 Plan d'Action et Timeline 21
9.1 Phase 1 : Fondation (Semaines 1-2) 21
9.2 Phase 2 : Contrats d'API (Semaine 3) 21
9.3 Phase 3 : PFE Pilote (Semaines 4-6) 22
9.4 Phase 4 : Intégration IA (Semaines 7-9) 22
9.5 Phase 5 : Intégration Finale & Tests (Semaines 10-12) 22

10 Standards et Bonnes Pratiques 24
10.1 Convention de Nommage 24
10.1.1 Base de Données (Prisma) 24
10.1.2 Code Backend (NestJS) 24
10.1.3 Code Frontend (Next.js) 24
10.2 Gestion des Erreurs 24
10.3 Sécurité 25
10.4 Tests 25
10.5 Documentation 25

11 Annexe : Communication entre Équipes 26
11.1 Réunions Recommandées 26
11.1.1 Par email(15 min, tous les jeudis) 26
11.1.2 Weekly Sync (1h, chaque Samedi) 26

Chapitre 1

Vision et Positionnement du Projet

1.1 Concept Central : L'Écosystème "Phygital"

TechKids Hub n'est pas simplement une plateforme d'e-learning ni un simple fabricant de kits éducatifs. C'est un **écosystème phygital** qui combine :

— **Physique** : Des kits électroniques concrets (Arduino, capteurs, LED)

— **Digital** : Une plateforme web/mobile interactive avec simulation et IA

— **Pédagogique** : Un parcours d'apprentissage progressif et personnalisé

1.2 Les Différenciateurs Clés

1. **Le Kit comme Point d'Entrée Physique** : L'enfant manipule de vrais composants, ce qui ancre l'apprentissage
2. **Le Labo Virtuel comme Extension** : Simulation en temps réel pour tester sans risque
3. **L'IA comme Tuteur Socratique** : Accompagnement personnalisé adapté au rythme de chaque enfant
4. **Le Suivi Parental Intelligent** : Tableaux de bord et rapports automatisés

1.3 User Journey Unifié

1.3.1 Pour le Parent

1. Découvre le kit sur la boutique en ligne
2. Achète ou loue le kit adapté à l'âge de son enfant
3. Reçoit le kit à domicile avec un QR code d'activation
4. Active le kit sur la plateforme TechKids Hub
5. Suit la progression de son enfant via le dashboard

1.3.2 Pour l'Enfant

1. Déballage du kit et découverte des composants

2. Connexion à la plateforme avec les identifiants parents
3. Parcours des modules théoriques interactifs
4. Expérimentation dans le Virtual Lab (simulation)
5. Construction du circuit physique guidé par l'IA
6. Validation et déblocage du niveau suivant

Chapitre 2

Architecture Globale

2.1 Stack Technique

2.1.1 Frontend

— **Framework** : Next.js 14+ (App Router)
— **UI Library** : Tailwind CSS + shadcn/ui
— **State Management** : Zustand ou React Context
— **WebSockets** : Socket.io-client (pour le Virtual Lab)

2.1.2 Backend

— **Framework** : NestJS (TypeScript)
— **Architecture** : Microservices modulaires
— **Base de Données** : PostgreSQL
— **ORM** : Prisma
— **Cache** : Redis (sessions temps réel)
— **Message Queue** : RabbitMQ ou Redis Streams

2.1.3 IA & Simulation

— **Framework** : Python FastAPI
— **LLM** : OpenAI GPT-4 ou Claude (via API)
— **Circuit Simulation** : PySpice ou équivalent
— **Orchestration** : LangChain pour le tutorat conversationnel

2.2 Architecture en Microservices

Architecture Distribuée

L'architecture est basée sur un modèle de microservices communiquant via une API Gateway centralisée. Chaque PFE (Projet de Fin d'Études) représente un microservice indépendant.

Composants principaux :

- API Gateway (NestJS) - Authentification JWT centralisée
- PFE 1 : E-Commerce & Gestion des Kits
- PFE 2 : Edu-Tracker (Suivi pédagogique)
- PFE 3 : Virtual Lab (Simulation)
- PFE 4 : Content Generator (Génération automatisée)
- PostgreSQL via Prisma ORM
- Redis pour le cache
- RabbitMQ pour la messagerie asynchrone
- Cerveau IA (Python/FastAPI)

2.3 Organisation en Monorepo

```
techkids-hub/
|-- apps/
| |-- web/ # Frontend Next.js
| |-- api-gateway/ # Point d'entree NestJS
| |-- ecommerce-service/
| |-- edu-tracker-service/
| |-- virtual-lab-service/
| `-- ai-brain/ # Service Python
|-- libs/
| |-- database/ # Schema Prisma partage
| |-- shared-types/ # Types TypeScript communs
| |-- auth/ # Guards et strategies JWT
| `-- ui-components/ # Composants React reutilisables
`-- package.json
```

Listing 2.1 – Structure du projet

Chapitre 3

Schéma de Base de Données

3.1 Domaine : Noyau (Users & Auth)

Le schéma utilise Prisma ORM pour la gestion de la base de données PostgreSQL. Voici les modèles principaux :

Modèle User

Le modèle User supporte quatre rôles : PARENT, STUDENT, TEACHER, et ADMIN. Chaque utilisateur peut avoir plusieurs étudiants associés (relation parent-enfant).

Enums définis :

- UserRole : PARENT | STUDENT | TEACHER | ADMIN
- KitCategory : BEGINNER | INTERMEDIATE | ADVANCED
- ItemStatus : AVAILABLE | RENTED | IN_MAINTENANCE
- OrderStatus : PENDING | PAID | SHIPPED | DELIVERED | CANCELLED
- ExerciseType : QUIZ | CIRCUIT_BUILD | CODE_CHALLENGE
- ProgressStatus : NOT_STARTED | IN_PROGRESS | COMPLETED
- SourceType : PDF | VIDEO | WEBPAGE
- DraftStatus : PENDING_REVIEW | APPROVED | REJECTED

3.2 Points Clés du Schéma

3.2.1 Flexibilité Kit vs PhysicalItem

Architecture Flexible

- **Kit** : Le produit "modèle" (ex : "Kit Arduino Débutant")
- **PhysicalItem** : L'instance physique (ex : "Kit #12345 avec QR code ABC")

Cette séparation permet la gestion de location sans dupliquer les données produit.

3.2.2 Workflow de Génération de Contenu

Le processus de génération suit ce flux :

ContentSource → **GeneratedDraft** (avec statut) → **Module** (après approbation)

Cela permet une validation humaine avant publication.

3.2.3 Progression Granulaire

— Table **Progression** : suivi par module et par étudiant
— Table **Submission** : historique de chaque tentative d'exercice

Chapitre 4

Répartition des Projets de Fin d'Études (PFE) et Spécifications IA

4.1 PFE 1 : Plateforme E-Commerce & Gestion des Kits

Périmètre :

- Catalogue de produits (kits)
- Panier d'achat et processus de paiement
- Gestion des commandes (achat et location)
- Gestion du stock de PhysicalItems
- Génération et gestion des QR codes

Endpoints Clés de l'API :

```
POST /api/orders # Creer une commande
GET /api/orders/:orderId # Detail d'une commande
POST /api/rentals # Louer un kit
PATCH /api/rentals/:id/return # Retourner un kit
GET /api/kits # Lister les kits
GET /api/kits/:id # Detail d'un kit
```

Listing 4.1 – API E-Commerce

Composants IA Intégrés :

- Système de recommandation de kits basé sur l'âge et les préférences.
- Chatbot pour assistance à l'achat.

4.1.1 Implémentation IA : Algorithmes et Bibliothèques

Tâche IA Clé : Recommandation de produits (kits).

Algorithme Recommandé : Filtrage Collaboratif via **SVD (Décomposition en Valeurs Singulières)**.

- **Pourquoi ?** C'est une approche éprouvée, efficace avec des données structurées (utilisateur, produit, note) et plus simple à expliquer et à mettre en œuvre que des modèles de Deep Learning complexes. Elle constitue une excellente base de départ (Minimum Viable Product).

Bibliothèques Clés (Python) :

- **scikit-learn** : Pour l'implémentation de `TruncatedSVD`.
- **pandas** : Pour la manipulation des données d'entraînement (historique des achats, notations).
- **surprise** : Une bibliothèque spécialisée dans les systèmes de recommandation qui simplifie grandement l'expérimentation avec des algorithmes comme SVD.

4.2 PFE 2 : Edu-Tracker (LMS & Suivi Pédagogique)

Périmètre :

- Gestion des cours, modules et exercices.
- Suivi de la progression des étudiants.
- Dashboard parent avec analytics.
- Système d'inscriptions (enrollments).

Endpoints Clés de l'API :

```
GET /api/students/:id/progression # Progression globale
GET /api/courses/:courseId/modules # Modules d'un cours
POST /api/progressions # Mettre a jour la progression
GET /api/parents/dashboard # Dashboard parent
```

Listing 4.2 – API Edu-Tracker

Composants IA Intégrés :

- Génération automatique de rapports de progression.
- Détection des zones de difficulté de l'enfant.
- Recommandations de modules adaptés.

4.2.1 Implémentation IA : Algorithmes et Bibliothèques

Tâche IA Clé : Classification des élèves pour détecter les difficultés.
Algorithme Recommandé : Random Forest ou **XGBoost**.

- **Pourquoi ?** Ces algorithmes sont extrêmement performants sur des données tabulaires (temps de résolution, taux d'erreur, nombre de tentatives). Leur grand avantage est l'**explicabilité** : ils permettent d'extraire l'importance des features pour dire précisément aux parents "votre enfant bloque sur les concepts de boucles car il passe trop de temps sur les exercices Y et Z".

Bibliothèques Clés (Python) :

- **scikit-learn** : Pour `RandomForestClassifier` et les outils de métriques (`classification_report`, `confusion_matrix`).
- **xgboost** : Pour une implémentation optimisée de XGBoost.
- **pandas** : Pour créer les features (ex : temps moyen par exercice, score moyen par module).

4.3 PFE 3 : Virtual Lab (Simulation & Tutorat IA)

Périmètre :

— Interface de simulation de circuits électroniques.
— Éditeur de code en temps réel.
— Communication bidirectionnelle via WebSocket.
— Tuteur IA conversationnel (méthode Socratique).
— Validation automatique des exercices.

Architecture Temps Réel :

Client (React) ↔ WebSocket ↔ SimulationGateway (NestJS)
↔ Redis (cache de session) ↔ HTTP/gRPC ↔ Cerveau IA (Python / FastAPI)

Flux de Travail :

1. L'étudiant ouvre un exercice → Création d'une session Redis.
2. Modifications du circuit/code → Envoi via WebSocket → Stockage dans Redis.
3. Demande d'aide → SimulationGateway → Cerveau IA → Réponse Socratique.
4. Soumission finale → Validation par l'IA → Enregistrement dans la base de données (PostgreSQL).

Composants IA Intégrés :

— Tutorat Socratique (guide l'étudiant par des questions sans donner la réponse).
— Validation automatique du circuit et du code.
— Détection des erreurs courantes et suggestions contextuelles.

4.3.1 Implémentation IA : Algorithmes et Bibliothèques

Tâche IA Clé : Génération de dialogue contextuel et analyse sémantique.

Algorithme Recommandé : LLM (Large Language Model) avec une architecture **RAG (Retrieval-Augmented Generation)**.

— **Pourquoi ?** Un LLM (comme Llama 3 ou Mistral) est indispensable pour comprendre la nuance du code ou du circuit et pour engager un dialogue flexible. Le RAG permet d'ancrer le modèle dans le contexte de l'exercice (énoncé, cours associés) pour éviter les hallucinations et fournir des aides pertinentes.

Bibliothèques Clés (Python) :

— **transformers** (Hugging Face) : Pour charger et utiliser les modèles LLM.
— **langchain** ou **llama-index** : Pour orchestrer le pipeline RAG (création des embeddings, recherche dans la base vectorielle, génération de la réponse).
— **FastAPI** : Pour exposer le "Cerveau IA" comme un microservice.
— **ChromaDB** ou **FAISS** : Pour la base de données vectorielle, essentielle pour le RAG.

4.4 PFE 4 : Génération de Contenu Pédagogique Automatisé

Périmètre :

— Upload de sources variées (PDF, vidéos YouTube, liens web).
— Extraction et parsing du contenu (OCR, transcription, scraping).
— Génération de modules pédagogiques par l'IA.
— Interface de validation pour les enseignants/administrateurs.

Pipeline de Traitement :

1. Upload d'une source (PDF/URL) → Extraction du texte/vidéo (Python).
2. Envoi au "Cerveau IA" → Génération d'une ébauche (draft) via un LLM.
3. Création d'un objet GeneratedDraft (statut : PENDING_REVIEW).
4. Validation humaine via une interface d'administration.
5. Si approuvé (APPROVED) → Création du Module final dans la base de données.

Endpoints Clés de l'API :

```
POST /api/content-sources # Uploader une nouvelle source
GET /api/drafts?status=PENDING_REVIEW # Lister les brouillons a valider
PATCH /api/drafts/:id/approve # Approuver un brouillon
POST /api/drafts/:id/publish # Publier en tant que Module
```

Listing 4.3 – API Content Generator

Composants IA Intégrés :

— Analyse sémantique du contenu source.
— Génération de modules structurés (théorie + exercices).
— Adaptation du contenu à différents niveaux d'âge.

4.4.1 Implémentation IA : Algorithmes et Bibliothèques

Tâche IA Clé : Génération de texte structuré à partir de sources non structurées.

Algorithme Recommandé : LLM "Instruction-Tuned" (ex : Llama 3-Instruct, Mixtral).

— **Pourquoi ?** Ces modèles sont spécifiquement entraînés pour suivre des instructions complexes. En leur fournissant un "prompt" bien conçu ("Génère un module pédagogique pour des 10-12 ans à partir de ce texte, en incluant une théorie et 3 exercices"), ils peuvent produire des résultats structurés et de haute qualité.

Bibliothèques Clés (Python) :

— **transformers** (Hugging Face) : Pour le modèle de génération.
— **langchain** : Pour structurer les prompts et chaîner les étapes du pipeline.
— **PyPDF2** ou **pdfplumber** : Pour extraire le texte des fichiers PDF.
— **youtube-transcript-api** : Pour obtenir la transcription des vidéos YouTube.
— **beautifulsoup4** : Pour le scraping de contenu web.

Chapitre 5

Méthodologie de Validation et d'Implémentation des Composants IA

Pour valider scientifiquement les PFE, il est crucial de ne pas se contenter d'implémenter l'IA, mais de prouver son efficacité. Cette section propose une approche structurée pour l'évaluation et la mise en œuvre technique.

5.1 Métriques d'Évaluation des Modèles

L'évaluation en IA va au-delà de la simple fonctionnalité. Voici les métriques adaptées à chaque type de tâche.

5.1.1 Pour les systèmes de recommandation (PFE 1)

— **RMSE (Root Mean Square Error)** : Mesure l'écart entre la note prédite par l'IA et la note réelle. Une valeur basse est meilleure.

— **Précision à K (Precision@K)** : Parmi les K produits recommandés (ex : K=5), combien ont été réellement cliqués ou achetés ?

5.1.2 Pour la prédiction des difficultés (PFE 2)

— **F1-Score** : Crucial car il équilibre la précision (éviter les fausses alertes) et le rappel (ne manquer aucun élève en difficulté).

— **Matrice de Confusion** : Permet de visualiser les "Faux Positifs" (l'IA signale une difficulté qui n'existe pas) et les "Faux Négatifs" (l'IA manque une difficulté réelle).

5.1.3 Pour le Tutorat et la Génération (PFE 3 & 4)

L'évaluation des modèles de langue (LLM) est plus complexe et souvent qualitative.

— **Scores BLEU / ROUGE** : Comparent le texte généré à un texte de référence (rédigé par un expert).

— **Évaluation Humaine (Human-in-the-loop)** : Un expert note la pertinence, la cohérence et l'utilité des réponses de l'IA sur une échelle (ex : 1 à 5).

5.2 Feuille de Route pour l'Implémentation Technique

Pour chaque PFE, suivez ce workflow standardisé en ingénierie de l'IA.

5.2.1 Étape 1 : Collecte et Préparation des Données (Data Engineering)

— **Nettoyage** : Supprimer les données aberrantes (ex : un exercice réussi en 1 seconde).

— **Feature Engineering** : Créer des variables pertinentes (ex : "ratio temps passé / difficulté de l'exercice").

5.2.2 Étape 2 : Entraînement et Sélection du Modèle (Model Selection)

— **Division des données** : 80% pour l'entraînement, 20% pour le test.

— **Comparaison** : Tester au moins deux algorithmes et comparer leurs performances sur l'ensemble de test.

5.2.3 Étape 3 : Intégration et Déploiement (Deployment)

— **API** : Encapsuler le modèle entraîné dans une API (ex : FastAPI).

— **Validation** : Utiliser des schémas (ex : Pydantic) pour garantir la conformité des données échangées.

5.2.4 Étape 4 : Monitoring et Boucle de Rétroaction (Feedback Loop)

— **Collecte du feedback** : Mettre en place des mécanismes (ex : bouton "Cette aide a-t-elle été utile ?").

— **Ré-entraînement** : Utiliser les retours pour affiner (fine-tuning) le modèle périodiquement.

Chapitre 6

Schéma de Base de Données Complet

6.1 Schéma Prisma Complet et Optimisé

6.2 Domaine : Noyau (Users & Auth)

```
enum UserRole
  PARENT
  STUDENT
  TEACHER
  ADMIN


model User
  id String @id @default(uuid())
  email String @unique
  passwordHash String
  role UserRole

  // Relations
  profile Profile?
  students Student[] @relation("ParentStudents")
  orders Order[]
  reviewedDrafts GeneratedDraft[] @relation("DraftReviewer")

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt


model Profile
  id String @id @default(uuid())
  userId String @unique
  user User @relation(fields: [userId], references: [id])
  firstName String
  lastName String
  phoneNumber String?
  address String?

  createdAt DateTime @default(now())
```

```
  updatedAt DateTime @updatedAt
```

Listing 6.1 – Modele Utilisateur et Profile

6.3 Domaine : E-Commerce (PFE 1)

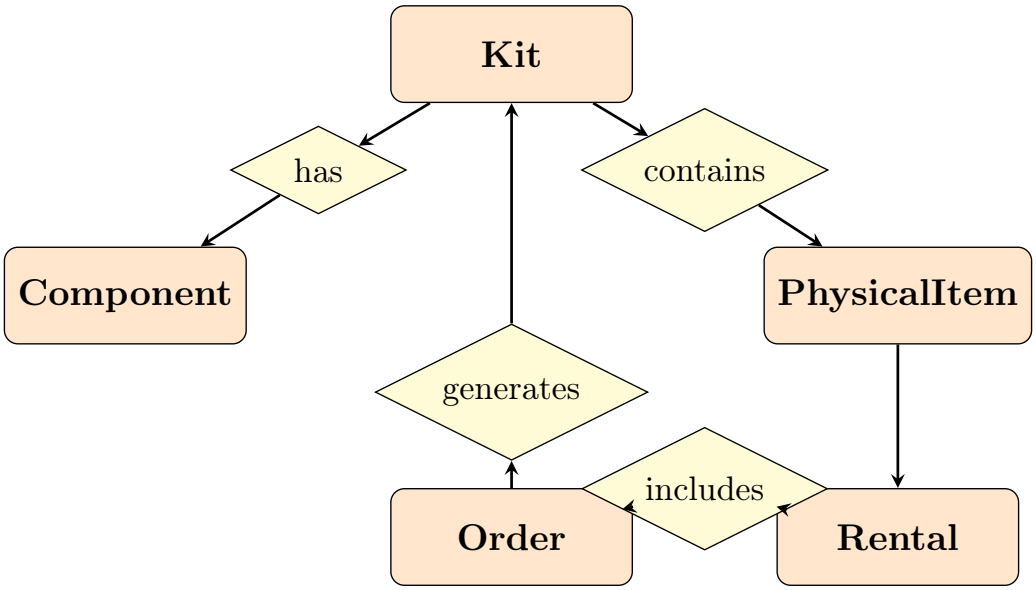

FIGURE 6.1 – Modèle relationnel du domaine E-Commerce

```
   enum KitCategory {
BEGINNER
INTERMEDIATE
ADVANCED
}

enum ItemStatus {
AVAILABLE
RENTED
MAINTENANCE
RETIRED
}

model Kit {
id String @id @default(uuid())
name String
description String
category KitCategory
recommendedAge Int
price Float
rentalPricePerMonth Float?
imageUrl String
```

```
// Relations
components Component[]
physicalItems PhysicalItem[]
courses Course[]

createdAt DateTime @default(now())
updatedAt DateTime @updatedAt
}

model PhysicalItem {
id String @id @default(uuid())
kitId String
kit Kit @relation(fields: [kitId], references: [id])
serialNumber String @unique
status ItemStatus @default(AVAILABLE)
qrCode String @unique

// Relations
rentals Rental[]

createdAt DateTime @default(now())
updatedAt DateTime @updatedAt
}
```

Chapitre 7

Points Clés du Schéma

7.1 Flexibilité Kit vs PhysicalItem

- **Kit** : Le produit "modèle" (ex : "Kit Arduino Débutant")
- **PhysicalItem** : L'instance physique (ex : "Kit #12345 avec QR code ABC")
- Cette séparation permet la gestion de location sans dupliquer les données produit

7.2 Workflow de Génération de Contenu

- **ContentSource** → **GeneratedDraft** (avec statut) → **Module** (après approbation)
- Permet une validation humaine avant publication

7.3 Progression Granulaire

- Table **Progression** : suivi par module et par étudiant
- Table **Submission** : historique de chaque tentative d'exercice

Chapitre 8

Répartition des PFE et Responsabilités

8.1 PFE 1 : E-Commerce & Gestion des Kits

8.1.1 Périmètre

- Catalogue de produits (kits)
- Panier d'achat et processus de paiement
- Gestion des commandes (achat et location)
- Gestion du stock de PhysicalItems
- Génération et gestion des QR codes

8.1.2 Endpoints Clés à Exposer

```
POST /api/orders # Crer une commande
GET /api/orders/:orderId # Dtail d'une commande
POST /api/rentals # Louer un kit
PATCH /api/rentals/:id/return # Retourner un kit
GET /api/kits # Liste des kits
GET /api/kits/:id # Dtail d'un kit
```

Listing 8.1 – API Endpoints PFE 1

8.1.3 IA Intégrée

- Recommandation de kits basée sur l'âge et les préférences
- Chatbot pour assistance à l'achat

8.2 PFE 2 : Edu-Tracker (LMS & Suivi Pédagogique)

8.2.1 Périmètre

- Gestion des cours, modules et exercices
- Suivi de la progression des étudiants
- Dashboard parent avec analytics

— Système d'inscriptions (enrollments)

Périmètre :

— Upload de sources (PDF, vidéos YouTube, liens web)
— Extraction et parsing du contenu (OCR, transcription, scraping)
— Génération de modules pédagogiques par l'IA
— Interface de validation pour les enseignants/admins

Pipeline de Traitement :

1. Upload de Source (PDF/URL) → Extraction texte/vidéo (Python)
2. Envoi au Cerveau IA → Génération de draft (prompt LLM)
3. Création d'un GeneratedDraft (status : PENDING_REVIEW)
4. Validation humaine via interface admin
5. Si APPROVED → Création du Module final dans la base

Endpoints Clés :

```
POST /api/content-sources # Upload nouvelle source
GET /api/drafts?status=PENDING_REVIEW # Drafts a valider
PATCH /api/drafts/:id/approve # Approuver un draft
POST /api/drafts/:id/publish # Publier en tant que Module
```

Listing 8.2 – API Content Generator

IA Intégrée :

— Analyse sémantique du contenu source
— Génération de modules structurés (théorie + exercices)
— Adaptation du contenu à différents niveaux d'âge

Chapitre 9

Plan d'Action et Timeline

9.1 Phase 1 : Fondation (Semaines 1-2)

Objectif : Mettre en place l'infrastructure commune
Tâches :

1. **Setup du Monorepo**
   - Initialiser Nx ou Turborepo
   - Configurer les workspaces (apps/ et libs/)
2. **Définir le Schéma Prisma**
   - Un référent technique crée `libs/database/prisma/schema.prisma`
   - Initialiser PostgreSQL (local ou via Docker)
   - Générer le client Prisma : `npx prisma generate`
3. **Authentification Centralisée**
   - Créer le module `libs/auth` avec JWT strategy
   - Développer les guards NestJS pour les rôles (PARENT, STUDENT, ADMIN)
   - Tester l'authentification avec Postman

Livrable

Infrastructure fonctionnelle + authentification opérationnelle

9.2 Phase 2 : Contrats d'API (Semaine 3)

Objectif : Définir les interfaces de communication entre microservices
Tâches :

1. **Documentation OpenAPI**
   - Chaque PFE documente ses endpoints avec Swagger/OpenAPI
   - Exemple : PFE 2 spécifie `GET /api/students/:id/progression`
2. **Définir les Événements Asynchrones**
   - Lister les événements inter-services (ex : `order.completed` → déclenche activation du cours)
   - Choisir RabbitMQ ou Redis Streams pour la message queue

Livrable

Spécifications OpenAPI validées par tous

9.3 Phase 3 : PFE Pilote (Semaines 4-6)

Objectif : Développer un PFE complet de bout en bout
PFE Recommandé : PFE 2 (Edu-Tracker) car il est au cœur du système
Tâches :

1. Implémenter les entités (Course, Module, Exercise, Progression)
2. Créer les endpoints REST
3. Développer l'interface frontend (liste des cours, détail d'un module)
4. Intégrer avec l'authentification
5. Tester la création et la mise à jour de progression

Livrable

Un étudiant peut s'inscrire à un cours, voir les modules, et sa progression est trackée

9.4 Phase 4 : Intégration IA (Semaines 7-9)

Objectif : Connecter le "Cerveau IA" aux PFE
Ordre d'Intégration Recommandé :

1. **PFE 4** : Génération de contenu (cas d'usage le plus simple)
   - API REST vers le service Python
   - Prompt simple : "Génère un module sur les LED pour des enfants de 10 ans"
2. **PFE 1** : Recommandations de kits
   - Appel API synchrone avec les préférences de l'utilisateur
3. **PFE 3** : Tutorat Socratique
   - WebSocket + gestion de contexte conversationnel
   - Intégration de LangChain pour le chaînage de prompts

Livrable

IA opérationnelle sur au moins 2 PFE

9.5 Phase 5 : Intégration Finale & Tests (Semaines 10-12)

Objectif : Connecter tous les PFE et tester les flux complets
Flux de Test Complet :

1. Un parent achète un kit (PFE 1)
2. Un cours est automatiquement activé (PFE 2)

3. L'enfant fait un exercice dans le Virtual Lab (PFE 3)
4. Un nouveau module est généré par l'IA (PFE 4)
5. Le dashboard parent affiche la progression (PFE 2)

Tâches :

— Tests d'intégration end-to-end
— Load testing (combien d'utilisateurs simultanés ?)
— Debugging des interactions entre services
— Documentation utilisateur

Chapitre 10

Standards et Bonnes Pratiques

10.1 Convention de Nommage

10.1.1 Base de Données (Prisma)

— Modèles : `PascalCase` (ex : `User`, `PhysicalItem`)
— Champs : `camelCase` (ex : `firstName`, `createdAt`)
— Relations : Pluriel pour "many" (ex : `students`, `orders`)

10.1.2 Code Backend (NestJS)

— Controllers : `nom-ressource.controller.ts`
— Services : `nom-ressource.service.ts`
— DTOs : `create-nom.dto.ts`, `update-nom.dto.ts`

10.1.3 Code Frontend (Next.js)

— Composants : `PascalCase` (ex : `StudentDashboard.tsx`)
— Hooks : `camelCase` avec préfixe `use` (ex : `useProgression.ts`)
— Pages : `kebab-case` (ex : `student-profile/page.tsx`)

10.2 Gestion des Erreurs

Backend :

```
// Utiliser les exceptions HTTP de NestJS
throw new NotFoundException(‘Kit with ID ${id} not found‘);
throw new BadRequestException(’Invalid age range’);
```

Listing 10.1 – Gestion des erreurs NestJS

Frontend :

```
// Toast notifications pour les erreurs utilisateur
try {
  await enrollInCourse(courseId);
  toast.success(’Inscription reussie’);
```

```
} catch (error) {
  toast.error('Erreur lors de l\'inscription');
}
```

Listing 10.2 – Gestion des erreurs frontend

10.3 Sécurité

Règles Obligatoires

1. **Jamais de données sensibles en clair** : Hash des mots de passe (bcrypt)
2. **Validation des entrées** : Utiliser `class-validator` pour les DTOs
3. **Guards sur tous les endpoints protégés** : `@UseGuards(JwtAuthGuard, RolesGuard)`
4. **CORS configuré** : Autoriser uniquement les domaines de confiance
5. **Rate limiting** : Limiter les requêtes par IP (ex : 100 req/min)

10.4 Tests

Types de Tests Requis :

— **Unitaires** : Chaque service et fonction critique

— **Intégration** : Tests des endpoints avec une base de test

— **E2E** : Scénarios utilisateur complets

Exemple de Test (Jest + NestJS) :

```
describe('ProgressionService', () => {
  it('should update progression correctly', async () => {
    const result = await service.updateProgression(
      studentId,
      moduleId,
      75
    );
    expect(result.completionRate).toBe(75);
  });
});
```

Listing 10.3 – Exemple de test unitaire

10.5 Documentation

Chaque PFE doit produire :

1. **README.md** : Comment lancer le service en local
2. **API.md** : Liste des endpoints avec exemples de requêtes
3. **ARCHITECTURE.md** : Schéma des flux de données

Chapitre 11

Annexe : Communication entre Équipes

11.1 Réunions Recommandées

11.1.1 Par email(15 min, tous les jeudis)

Chaque membre partage :

— Ce qui a été fait
— Ce qui bloque
— Ce qui est prévu

11.1.2 Weekly Sync (1h, chaque Samedi)

— Revue des contrats d'API
— Décisions d'architecture communes
— Démos inter-équipes